JavaScript

DÉVELOPPEMENT WEB : CONTRÔLE CÔTÉ UTILISATEUR

Dr Pascal FAYE

Introduction

- JavaScript permet de rendre dynamique le contrôle des actions utilisateur sur une page développé en HTML avant « submit » au serveur.

- Le JavaScript est un langage de script simplifié orienté objet dont la syntaxe est basée sur celle du Java.

Introduction

- Contrairement à un applet Java qui est un programme compilé, les scripts écrits en Javascript sont **interprétés**
  - Le Java, représenté par un ou plusieurs fichiers autonomes dont l'extension sera *.*class* ou *.*jar*, est invoqué par une balise HTML spécifique
  - Le JavaScript est **écrit directement au sein du document HTML** sous forme d'un script encadré par des balises HTML.

HTML et JavaScript

- Le code JavaScript s'intègre de deux manière avec le code HTML
  - 1. **Insertion directe** dans le code HTML
    - Le code JavaScript s'insère le plus souvent dans la page HTML.
    - C'est la méthode la plus simple et la plus fréquemment utilisée par les développeurs de sites Internet.

```
<script language="JavaScript">
.............
</script>
```

  - 2. **Insertion comme un module externe**

```
<script type="module" src=",,,,,,">
.............
</script>
```

1. Insertion dans une page HTML

- Le code s'exécute automatiquement lors du chargement de la page HTML dans le navigateur en même temps que le contenu de la page HTML s'affiche à l'écran. .

```
<html>
<head>
<title>..... </title>
</head>
<body>
  <script type="text/javascript">
        .
        .
  </script>
</body>

</html>
```

2. Insertion par appel de module externe

- On peut insérer du code JavaScript en faisant appel à un module externe se trouvant à n'importe quelle adresse (URI).

```
<script src="URL du module externe">
.............
</script>
```

- Les deux balises de Javascript doivent être placés entre les Tags **<body>** et **</body>** dans le cas d'une exécution directe ou entre les Tags **<head>** et **</head>** de la page HTML pour une exécution différée.
- Stocké dans un fichier sur le serveur à son adresse d'appel sous forme de **TEXTE SIMPLE** portant l'extension **.txt** ou **.js**
- Simplifie la maintenance des sites faisant appel à des modules JavaScript communs à plusieurs pages HTML.
- Inconvénient : l'appel au code externe génère une requête supplémentaire vers le serveur, et encombre le réseau

La structure d'un script en JavaScript

- La syntaxe du langage JavaScript s'appuie sur le modèle de Java et C
- **Règles générales**
  - Chaque commande doit être terminée par un point-virgule (;).
  - Un nombre à virgule est séparé par un point (.) et non par une virgule
  - Le langage JavaScript est **sensible à la casse**
  - Il existe deux méthodes permettant d'intégrer des commentaires à vos scripts.
    - Placer un double slash (//) devant le texte
    - Encadrer le texte par un slash suivi d'une étoile (/*) et la même séquence inversée (*/)

Exemple

```
<html>
  <head>
    <title>JavaScript</title>
    <script type="text/javascript">
    // Ici on definit une fonction JavaScript
    function direBonjour() {
      alert("USSEIN Prog Web !!! ")
    }
    </script>
  </head>
  <body>
    <div>
      <h1>Bonjour avec JavaScript</h1>

      <script type="text/javascript">
      // Ici on apelle la fonction
        direBonjour()
      </script>
    </div>
  </body>
</html>
```

Exemple

```
<html>
 <head>
  <title>JavaScript</title>
  <script type="text/javascript">
    function direBonjour() {
        alert("USSEIN Prog Web !!! ")
      }
    function tuascliquer() {
        alert("Tu clic et tu me vois !!! ")
      }
  </script>
 </head>
 <body>
 <form method="post">
  <div>
   <h1>Bonjour avec JavaScript</h1>
   <script type="text/javascript">
       direBonjour()
   </script>
  </div>
<input type="button" value="Clickez ici" onclick="tuascliquer()"/>
</form>
 </body>
</html>
```

Entrée et sortie de données avec JavaScript

- 3 types de boites de messages peuvent être affichés en utilisant Javascript : **Alerte, Confirmation et Invite**
  - **Méthode alert()**
    - sert à afficher à l'utilisateur des informations simples de type texte. Une fois que ce dernier a lu le message, il doit cliquer sur OK pour faire disparaître la boîte
  - **Méthode confirm()**
    - permet à l'utilisateur de choisir entre les boutons OK et Annuler.
  - **Méthode prompt()**
    - La méthode prompt() permet à l'utilisateur de taper son propre message en réponse à la question posée
- La méthode **document.write** permet d 'écrire du code HTML dans la page WEB

Entrée et sortie de données avec JavaScript

```
<html>
<head>
<title> une page simple </title>
</head>
<body>
Bonjour
<script language='javascript'>
alert('bonjour');
document.write (
  prompt('quel est votre nom ?','Indiquer votre nom ici')
  );
confirm(‘Faites votre choix!!!!');
</script>
</body>
</html>
```

Les variables (1)

- **Déclaration et affectation**

  - Le mot-clé **var** permet de déclarer une ou plusieurs variables (obsolète).
  - **let** : pour des variables dont la valeur peut changer.
  - **const** : pour des variables dont la valeur ne peut pas être modifiée après l'initialisation.
  - Après la déclaration de la variable, il est possible de lui affecter une valeur par l'intermédiaire du signe d'égalité (=).
  - Si une valeur est affectée à une variable sans que cette dernière ne soit déclarée, alors Javascript la déclare automatiquement.

```
//Déclaration de i, de j et de k. let est de nos jours le plus utilisé
let i, j, k;

 //Affectation de i.
i = 1;
//Déclaration et affectation de prix.
let prix = 0;

//Déclaration et affectation de caractere
let caractere = ["a", "b", "c"];
```

Les variables (2)

- **Déclaration et affectation**
  - La lecture d'une variable non déclarée provoque une erreur
  - Une variable correctement déclarée mais dont aucune valeur n'est affectée, est indéfinie (undefined).
- **La portée**
  - les variables peuvent être globales ou locales.
  - Une variable globale est déclarée en début de script et est accessible à n'importe quel endroit du programme.
  - Une variable locale est déclarée à l'intérieur d'une fonction et n'est utilisable que dans la fonction elle-même.

Les variables (3)

- **Contraintes concernant les noms de variables**
  - Les noms de variables ne peuvent contenir que des lettres, chiffres, ou le caractère "_" (underscore)
    - *Mon_Prenom* est un nom valide
  - Les caractères spéciaux et accentués sont interdits (é, à, ç, ï, etc..)
    - *Mon_Prénom* n'est pas un nom valide. Il y a un caractère accentué.
  - Les majuscules et les minuscules ont leur importance.
    - *MonPrenom* est différent de *Monprenom*.
  - Un nom de variable ne peut contenir d'espaces.
    - *Mon Prenom* n'est pas un nom de variable correct. Il y a un espace.
  - Les mots réservés JavaScript ne peuvent être utilisés comme noms de variable.

Les variables (4)

Les types de données les plus courants sont :

- string (chaîne de caractères) : "Hello"
- number (nombre) : 10, 3.14
- boolean (booléen) : true, false
- array (tableau) : [1, 2, 3]
- object (objet) : {nom: "Alice", age: 25}

Les variables (5)

- Le type d'une variable dépend de la valeur stockée dans cette variable. **Pas de déclaration de type.**
  - Exemple
    let maVariable = ' Philippe ';
    maVariable =10;

- trois principaux types de valeurs
  - String
  - Number : 10^-308 >nombre < 10^308
    - Les nombres entiers
    - les nombres décimaux en virgule flottant
    - 3 valeurs spéciales :
      - Positive Infinity ou +Infinity (valeur infini positive)
      - Negative Infinity ou –Infinity (valeur infinie négative)
      - Nan (Not a Number) habituellement générée comme résultat d'une opération mathamatique incohérente
  - Boolean
    - deux valeurs littérales : true (vrai) et false (faux).

Valeurs spéciales

- JavaScript inclut aussi deux types de données spéciaux :
  - **Null :** possède une seule valeur, **null**, qui signifie l'absence de données dans une variable
  - **Undefined :** possède une seule valeur, **undifined**. Une variable dont le contenu n'est pas clair car elle n'a jamais stocké de valeur, pas même **null** est dite non définie (undifined).

Les opérations sur les chaînes

- La concaténation
  - let chaine = "bonjour " + " FI3/FCD1";
- Déterminer la longueur d'une chaîne
  - let ch1 = "bonjour";
  - let longueur = ch1.length;
- Identifier le nième caractère d'une chaîne
  - let ch1 =" Rebonjour ! ";
  - let carac = ch1.charAt(2);
- Extraction d'une partie de la chaîne
  - let dateDuJour = "04/04/03"
  - let mois = datteDuJour.substring(3, 5);
    - 3: est l'indice du premier caractère de la sou-chaîne à extraire
    - 5 : indice du dernier caractère à prendre en considération ; ce caractère ne fera pas partie de la sous-chaîne à extraire

Les fonctions prédéfinies (1)

- **eval**
  - Cette fonction exécute un code Javascript à partir d'une chaîne de caractères.

```
 ...
<SCRIPT LANGUAGE="JavaScript">
function evaluation()  {
document.formulaire.calcul.value=eval(document.formulaire.saisie.value);  }
</SCRIPT>
...
<FORM NAME="formulaire">
Saisissez une expression mathématique : <INPUT TYPE="text" NAME=saisie MAXLENGTH=40 SIZE=40>
<INPUT TYPE="button" VALUE="evaluation." onClick="evaluation()">
<INPUT TYPE="text" NAME=calcul MAXLENGTH=40 SIZE=40>
</FORM>...
```

Les fonctions prédéfinies (2)

- **isFinite**
  - Détermine si le parametre est un nombre fini. Renvoie *false* si ce n'est pas un nombre ou l'infini positif ou infini négatif.

```
isFinite(240) //retourne true
isFinite("Un nombre") //retourne false
```

- **isNaN**
  - détermine si le parametre n'est pas un nombre (NaN : Not a Number).

```
isNaN("un nombre") //retourne true
isNaN(20) //retourne false
```

Les fonctions prédéfinies (3)

- **parseFloat**
  - analyse une chaîne de caractères et retourne un nombre décimal.
  - Si l'argument évalué n'est pas un nombre, renvoie *NaN* (Not a Number).

```
let numero="125";
let nombre=parseFloat(numero); //retourne le nombre 125
```

- **parseInt**
  - analyse une chaîne de caractères et retourne un nombre entier de la base spécifiée.
  - La base peut prendre les valeurs *16* (hexadécimal) *10* (décimal), *8* (octal), *2* (binaire).

```
let prix=30.75;
let arrondi = parseInt(prix, 10); //retourne 30
```

Les fonctions prédéfinies (4)

Number

```
let jour = new Date("December 17, 1995 03:24:00");//converit la date en millisecondes
alert (Number(jour));
```

String

- convertit l'objet spécifié en chaîne de caractères

```
jour = new Date(430054663215);//Convertit le nombre en date Mois jour, Annéee etc.
alert (String(jour));
```

Escape

- retourne la valeur hexadécimale à partir d'une chaîne de caractère codée en ISO-Latin-1.

```
escape("!&") //retourne %21%26%
```

Les événements (1)

- Javascript est dépendant des événements
  - se produisent lors d'actions diverses sur les objets d'un document HTML.
    - onLoad;
    - onClick
    - onMouseover
    - onMouseout
    - ...
- Il est possible de baser l'exécution de fonctions sur des événements

Les événements (2)

Evénement onLoad

- Se produit lorsque une page web est chargée dans la fenêtre du navigateur
- Toute la page (y compris les images qu'elle contient si leur chargement est prévu) doit avoir été chargée pour qu'il ait lieu
- Cet événement peut être associé à une image seulement ; auquel cas, il se produit une fois son chargement terminé

```
<HTML><BODY onLoad="alert('page chargée');">
Exemple de l'événement onLoad
</BODY></HTML>
```

Les événements (3)

Evénement onClick

- Se produit lorsque l'utilisateur clique sur un élément spécifique dans une page, comme un lien hypertexte, une image, un bouton, du texte, etc.
- Ces éléments sont capables de répondre séparément à cet événement
- Il peut également être déclenché lorsque l'utilisateur clique n'importe où sur la page s'il a été associé non pas à un élément spécifique, mais à l'élément body tout entier

```
<HTML><BODY>
<INPUT TYPE="Button" Value="cliquer ici”
onClick="alert('Clic')">
</BODY></HTML>
```

Les événements (4)

Événement onMouseover

- Analogue à onClick sauf qu'il suffit que l'utilisateur place le pointeur de sa souris sur l'un des éléments précités (lien hypertexte, image, bouton, texte, etc.) pour qu'il ait lieu

Événement onMouseout

- A l'inverse de onMouseover, cet événement se produit lorsque le pointeur de la souris quitte la zone de sélection d'un élément.

```
<HTML><BODY>
<IMG SRC="image.gif" onMouseOver="src='image2.gif';"
onMouseOut="src='image.gif';">
</BODY></HTML>
```

Les Objets (1)

- Les objets de JavaScript, sont soit des entités pré définies du langage, soit créés par le programmeur.
  - Par exemple, le navigateur est un objet qui s'appelle "**navigator**".
  - La fenêtre du navigateur se nomme "**window**"
  - La page HTML est un autre objet, que l'on appelle "**document**".
  - Un formulaire à l'intérieur d'un "**document**", est aussi un objet.
  - Un lien hypertexte dans une page HTML, est encore un autre objet. Il s'appelle "**link**". etc...
- Les objets javascript peuvent réagir à des "Evénements".
- Tous les navigateurs ne supportent pas les mêmes objets
- Accès aux propriétés d'un objet
  - voiture.couleur.value
  - voiture.couleur.value = verte

Les objets (2)

- **L'opérateur New**
  - L'opérateur *new* est utilisé pour créer une nouvelle instance ou un nouveau type d'objet défini par l'utilisateur ou de l'un des types d'objets prédéfinis, *Array*, *Boolean*, *Date*, *Function*, *Image*, *Number*, *Object*, ou *String*.
    - **nouvel_objet = new type_objet(parametres)**

```
texte = new String("Une chaîne de caractère");
```

Les objets (3)

L'opérateur Typeof

- L'opérateur *typeof* renvoie une chaîne de caractères indiquant quel est le type de l'opérande.

```
let i = 1;
typeof i; //retourne number
let titre="Les raisins de la colère";
typeof titre; //retourne string
let jour = new Date();
typeof jour; //retourne object
let choix = true;      typeof choix; //retourne boolean
let cas = null;       typeof cas; //retourne object
typeof parseFloat; //retourne function
typeof Math; //retourne object (IE 5.*, NS 6.*, NS 4.78, Opera 6.*, Opera 5.*
typeof Math; //retourne function NS 3.*, Opera 3.*
```

L'objet *String (1)*

- Propriété :
  - *length* : retourne la longueur de la chaîne de caractères;
- Méthodes :
  - *anchor( )* : formate la chaîne avec la balise <A> nommée;
  - *b( )* : formate la chaîne avec la balise <B>;
  - *big( )* : formate la chaîne avec la balise <BIG>;
  - *charAt( )* : renvoie le caractère se trouvant à une certaine position;
  - *charCodeAt( )* : renvoie le code du caractère se trouvant à une certaine position;
  - *concat( )* : permet de concaténer 2 chaînes de caractères;
  - *fromCharCode( )* : renvoie le caractère associé au code;
  - *indexOf( )* : permet de trouver l'indice d'occurrence d'un caractère dans une chaîne;

L'objet *String* (2)

- *italics( )* : formate la chaîne avec la balise <I>;
- *lastIndexOf( )* : permet de trouver le dernier indice d'occurrence d'un caractère;
- *link( )* : formate la chaîne avec la balise <A> pour permettre de faire un lien;
- *slice( )* : retourne une portion de la chaîne;
- *substr( )* : retourne une portion de la chaîne;
- *substring( )* : retourne une portion de la chaîne;
- *toLowerCase( )* : permet de passer toute la chaîne en minuscule;
- *toUpperCase( )* : permet de passer toute la chaîne en majuscules;

L'objet *Array*

- Propriété :
  - *length* : retourne le nombre d'éléments du tableau;
- Méthodes :
  - *concat( )* : permet de concaténer 2 tableaux;
  - *join( )* : converti un tableau en chaîne de caractères;
  - *reverse( )* : inverse le classement des éléménts du tableau;
  - *slice( )* : retourne une section du tableau;
  - *sort( )* : permet le classement des éléments du tableau;

L'objet *Math (1)*

- Propriétés :
  - *E* : renvoie la valeur de la constante d'Euler (~2.718);
  - *LN2* : renvoie le logarithme népérien de 2 (~0.693);
  - *LN10* : renvoie le logarithme népérien de 10 (~2.302);
  - *LOG2E* : renvoie le logarithme en base 2 de e (~1.442);
  - *LOG10E* : renvoie le logarithme en base 10 de e (~0.434);
  - *PI* : renvoie la valeur du nombre pi (~3.14159);
  - *SQRT1_2* : renvoie 1 sur racine carrée de 2 (~0.707);
  - *SQRT2* : renvoie la racine carrée de 2 (~1.414);

L'objet *Math (2)*

- Méthodes :
  - *abs( ), exp( ), log(), sin( ), cos( ), tan( ), asin( ), acos( ), atan( ), max( ), min( ), sqrt( )* sont les opérations mathématiques habituelles;
  - *atan2( )* : retourne la valeur radian de l'angle entre l'axe des abscisses et un point;
  - *ceil( )* : retourne le plus petit entier supérieur à un nombre;
  - *floor( )* : retourne le plus grand entier inférieur à un nombre;
  - *pow( )* : retourne le résultat d'un nombre mis à une certaine puissance;
  - *random( )* : retourne un nombre aléatoire entre 0 et 1;
  - *round( )* : arrondi un nombre à l'entier le plus proche.

L'objet *Date (1)*

- Propriété : aucune;
- Méthodes :
  - *getFullYear( ), getYear( ), getMonth( ), getDay( ), getDate(), getHours( ), getMinutes( ), getSeconds( ), getMilliseconds( ):* retournent respectivement l'année complète, l'année (2chiffres), le mois, le jour de la semaine, le jour du mois, l'heure, les minutes, les secondes et les millisecondes stockés dans l'objet *Date*;
  - *getUTCFullYear( ), getUTCYear( ),* … retournent respectivement l'année complète, l'année (2chiffres), … stockés dans l'objet *Date* en temps universel;
  - *setFullYear( ), setYear( ),* … remplacent respectivement l'année complète, l'année (2chiffres), … dans l'objet *Date;*

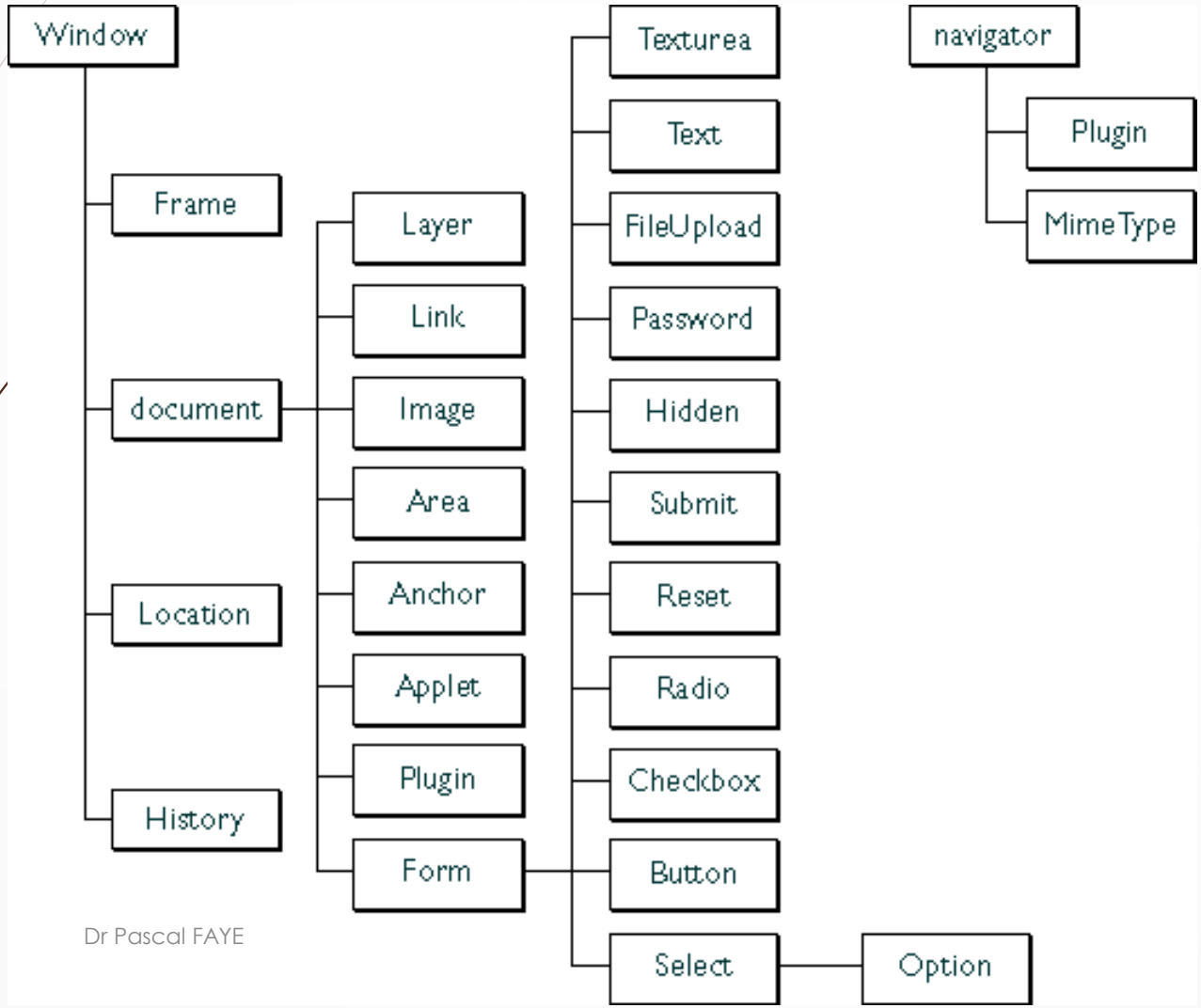
L'objet *Date (2)*

- *setUTCFullYear( ), setUTCYear( ),* … remplacent l'année complète, l'année (2chiffres), … dans l'objet *Date* en temps universel;
- *getTime( )* : retourne le temps stocké dans l'objet *Date*;
- *getTimezoneOffset( )* : retourne la différence entre l'heure du client et le temps universel;
- *toGMTString( ), toLocaleString( ), toUTCString( )* : convertissent la date en chaîne de caractère selon la convention GMT, selon la convention locale ou en temps universel;

Les objets du navigateur (1)

- L'objet le plus haut dans la hiérarchie est *window* qui correspond à la fenêtre même du navigateur.
- L'objet *document* fait référence au contenu de la fenêtre.
- *document* regroupe l'ensemble des éléments HTML présents sur la page. Pour atteindre ces différents éléments, nous utiliserons :
  - soit des méthodes propres à l'objet *document,* comme la méthode *getElementById( ),* qui permet de trouver l'élément en fonction de son identifiant (ID);
  - soit des collections d'objets qui regroupent sous forme de tableaux Javascript tous les éléments de type déterminé.

Les objets du navigateur (2)



L'objet *window (1)*

- Propriétés : (accessibles avec IE et N)
  - *closed* : indique que la fenêtre a été fermée;
  - *defaultStatus* : indique le message par défaut dans la barre de status;
  - *document* : retourne l'objet *document* de la fenêtre;
  - *frames* : retourne la collection de cadres dans la fenêtre;
  - *history* : retourne l'historique de la session de navigation;
  - *location* : retourne l'adresse actuellement visitée;
  - *name* : indique le nom de la fenêtre;

L'objet *window* (2)

- *navigator* : retourne le navigateur utilisé;
- *opener* : retourne l'objet *window* qui a créé la fenêtre en cours;
- *parent* : retourne l'objet *window* immédiatemment supérieur dans la hiérarchie;
- *self* : retourne l'objet window correspondant à la fenêtre en cours;
- *status* : indique le message affiché dans la barre de status;
- *top* : retourne l'objet *window* le plus haut dans la hiérarchie.

L'objet *window* (3)

- Méthodes :
  - *blur( )* : enlève le focus de la fenêtre;
  - *close( )* : ferme la fenêtre;
  - *focus( )* : place le focus sur la fenêtre;
  - *moveBy( )* : déplace d'une distance;
  - *moveTo( )* : déplace la fenêtre vers un point spécifié;
  - *open( )* : ouvre une nouvelle fenêtre;
  - *print( )* : imprime le contenu de la fenêtre;
  - *resizeBy( )* : redimensionne d'un certain rapport;
  - *resizeTo( )* : redimensionne la fenêtre;
  - *setTimeout( )* : évalue une chaîne de caractère après un certain laps de temps.

L'objet *document (1)*

- Propriétés :
  - *applets* : retourne la collection d'applets java présente dans le document;
  - *cookie* : permet de stocker un cookie;
  - *domain* : indique le nom de domaine du serveur ayant apporté le document;
  - *forms* : retourne la collection de formulaires présents dans le document;
  - *images* : retourne la collection d'images présentes dans le document;
  - *links* : retourne la collection de liens présents dans le document;

L'objet *document (2)*

- *referrer* : indique l'adresse de la page précédente;
- *title* : indique le titre du document.

- Méthodes :
  - *close( )* : ferme le document en écriture;
  - *open( )* : ouvre le document en écriture;
  - *write( )* : écrit dans le document;
  - *writeln( )* : écrit dans le document et effectue un retour à la ligne

L'objet *navigator*

- Propriétés
  - *appName* : application (Netscape, Internet Explorer)
  - *appVersion* : numero de version.
  - *platform* : système d'exploitation (Win32)
  - *plugins*
  - *language*
  - *mimeTypes*
  - *JavaEnabled()*

Nommage des objets-éléments

- Pour pouvoir manipuler un objet en javaScript, il doit posséder un nom
- Pour pouvoir distinguer les différents objets-éléments d'une page web, il suffit de leur donner un nom à travers de l'attribut NAME
  - <Table Name=« tableau1 »>…
  - <Table Name=« tableau2 »>…
  - <Form Name = « formulaire1 »>…
  - <Form Name =« formulaire2 »>…
  - <Textarea Name =« texte1»>…
- Dans le cas où l'objet serait unique alors pas besoin de nom pour désigner cet objet
  - Exemple : le cas de BODY(une seul BODY par document), DOCUMENT (un seul DOCUMENT par fenêtre)

Manipulation des objets

- Pour adresser un objet, il ne suffit pas de donner son nom : il faut aussi préciser son « chemin d'accès » dans l'arborescence de la structure

```
<HTML><BODY
onLoad="window.document.formulaire.zone.value='Bonjour';">
<FORM name="formulaire"><INPUT NAME="zone" TYPE="text">
</FORM></BODY></HTML>
```

- Si le nom de la fenêtre est omis, le navigateur utilisera par défaut la fenêtre courante
- Dans le cas de cadres (frames), il est pertinent de donner le nom de la fenêtre
- Il est possible aussi d'omettre window.document : l'adressage réussi puisqu'il n'y a qu'un seul objet « document» dans la fenêtre

Structures de contrôle

- Conditionnelle if/else

```
let age = 18;
if (age >= 18)
{
    console.log("Vous êtes majeur");
} else {
console.log("Vous êtes mineur");
}
```

Structures de contrôle

- Boucles for et while

```
for (let i = 0; i < 5; i++) {
 console.log(i);
}
let i = 0;
while (i < 5) {
 console.log(i);
i++;
}
```

Exemple de fonction utilisateur

```
function saluer(nom)
{
 console.log("Bonjour, " + nom);
}



saluer("Pascal");  // Appelle de la fonction
```

Exemple de fonction utilisateur

```
setTimeout(function() {
    console.log("Message après 2 secondes");
}, 2000);  // 2000 ms = 2 secondes


let compteur = 0;
setInterval(function() {
    console.log(compteur);
    compteur++;
}, 1000);  // 1000 ms = 1 seconde
```

Exemple de fonction utilisateur

```
let promesse = new Promise(function(resolve, reject) {
    let succes = true;
    if (succes) {
        resolve("Opération réussie !");
    } else {
        reject("Opération échouée !");
    }
});
promesse.then(function(resultat) {
    console.log(resultat);  // Affiche "Opération réussie !"
}).catch(function(error) {
    console.log(error);     // Affiche "Opération échouée !"
});
```

Exemple de fonction utilisateur

```
async function exempleAsync() {
    try {
        let resultat = await promesse;
        console.log(resultat);
    } catch (error) {
        console.log(error);
    }
}

exempleAsync();
```

Manipulation du DOM (Document Object Model)

- JavaScript permet de contrôler le contenu de la page web.

```
<script language="javascript">
function verification(){
   if(document.getElementById('nom').value == "" | |
document.getElementById('pnom').value == "" | |
document.getElementById('age').value == "" | |
document.getElementById('numCarte').value == "" {
      alert ('Vous avez oublié de renseigner un ou des champs obligatoires !!!!');
      document.getElementById('numCarte').value="";
      return false;
   }
return true;
}
</script>
```

Manipulation du DOM (Document Object Model)

- JavaScript permet de contrôler le contenu de la page web.

```
<script language="javascript">
function chiffres(event) {
    // Compatibilité IE / Firefox
    if(!event&&window.event) { event=window.event; }
    // IE
    if((event.which < 48 || event.which > 57 )  &&  event.keyCode !=8  &&  event.keyCode !=9) {
        event.returnValue = false;
        event.cancelBubble = true;
    }
    // DOM
    if( (event.which < 48 || event.which > 57 ) && event.keyCode !=8  && event.keyCode !=9) {
        event.preventDefault();
        event.stopPropagation();
    }}
</script>
```

Manipulation du DOM (Document Object Model)

```
<FORM method="POST" action="ins_etudiant.php" onsubmit='return verification();'>

<table align ="center" width="495" height="338">

<tr><td width="143"><font color="blue"><b>Numéro Carte</b></font></td><td width="147">
<input type="text" name="numCarte" id='nom'><font color="red">*</font></td></tr>

  <tr><td width="143"><font color="blue"><b>Prenom</b></font></td><td width="147"><input
type="text" name="pnom" id='pnom'><font color="red">*</font></td></tr>

   <tr><td width="143"><font color="blue"><b>Nom</b></font></td><td width="147">  <input
type="text" name="nom" id='nom'><font color="red">*</font></td></tr>

 <tr><td width="143"><font color="blue"><b>Age</b></font></td> <td width="147"><input
type="text" name="age" id='age' onkeypress="chiffres(event)" maxlength="2"><font
color="red">*</font> <font color="blue">ans</font></td></tr>

<tr><td><input type="submit" name="val" value="Valider"></td><td><input type="reset"
name="an" value="Effacer"></td></tr>

  </table>

   </form>

<h4><font color="red">*</font>  <font color="black">Zone de saisie obligatoire</font></h4>
```

Collections (objet et tableau)

```
let personne = {
    nom: "Pascal",
    age: 25,
    saluer: function() {
        console.log("Bonjour, " + this.nom);
    }
};

console.log(personne.nom);  // Affiche "Pascal"
personne.saluer();          // Appelle la méthode saluer
```

Collections (objet et tableau)

```
let fruits = ["Pomme", "Banane", "Orange"];

console.log(fruits[0]); // Affiche "Pomme«

fruits.push("Fraise");  // Ajoute "Fraise" à la fin

console.log(fruits);    // Affiche ["Pomme", "Banane", "Orange", "Fraise"]
```

TP

FAQ